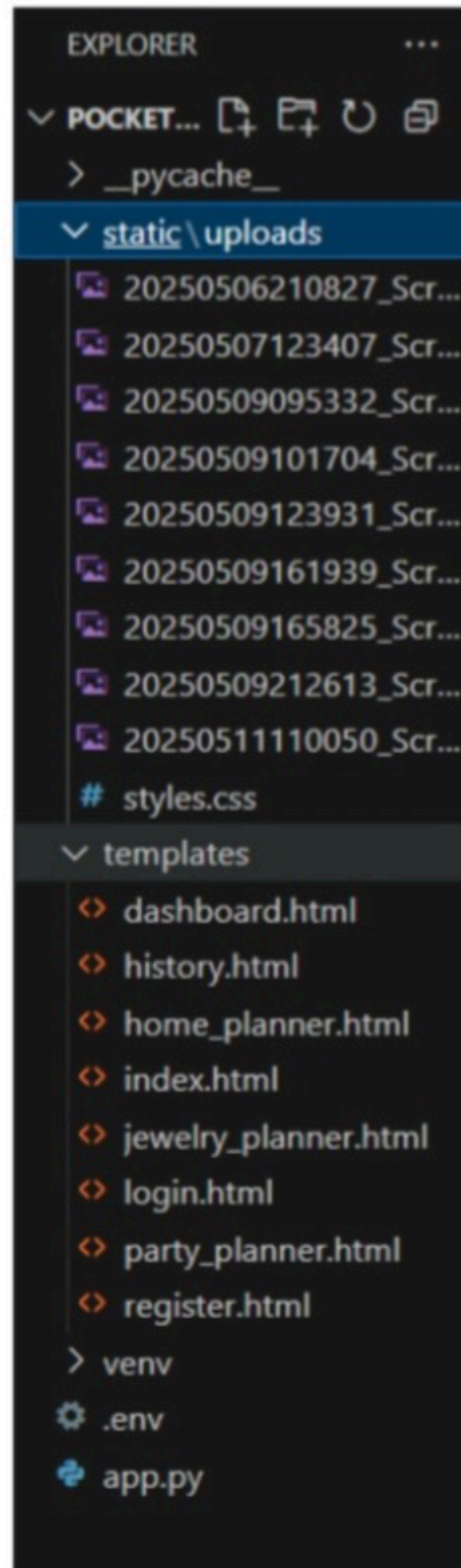


MILESTONE 2: Core Functionalities Development :

- **Framework & Model:** Built on Flask leveraging Google's Gemini 1.5 Flash Pro foundation model.
 - **Core Inputs:** Accepts user budgets and optional multimodal image inputs.
 - **Domain Segmentation:** Uses modular planning logic tailored for three specific categories: Home, Party, and Jewelry.
 - **Centralized Logic:** Abstracted into a utility file (gemini_utils.py) to manage prompt orchestration, budget formatting, and image analysis.
 - **System Resilience:** Implements fallback strategies to maintain recommendations when product matches are limited.
-

File Explorer Structure



Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

```
import os
import json
import re
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

- Fast API Initialization

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

- Load .env environment

```
# Load environment variables
load_dotenv()
# Configure Google Gemini AI - Do this once, with consistent environment variable name
API_KEY = os.getenv("GOOGLE_API_KEY")
if not API_KEY:
    # Try alternative environment variable name
    API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:
    raise ValueError("No Google API key found in environment variables. Please set GOOGLE_API_KEY in your .env file.")

genai.configure(api_key=API_KEY)

# Password hashing context
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False) # Allow optional token

class RegisterUser(BaseModel):
    username: str
    email: str
    full_name: Optional[str] = None
    password: str

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Adjust in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Static files and templates
templates = Jinja2Templates(directory="templates")
os.makedirs("static/uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Activity 2.2: Main Functionality code routings

- **/generate-home:** Generates home interior recommendations (furniture, decor, lighting)

```
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """Generate home interior recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need interior design product recommendations for a home in India with a total budget of ₹{budget_input.total_budget:.2f}.
        Requirements:
        - {budget_input.num_lights} lights/lighting fixtures
        - {budget_input.num_fans} ceiling fans
        - {budget_input.num_furniture} furniture pieces
        - {budget_input.num_dining_tables} dining tables

        Additional rooms to consider:
        {("- Living room" if budget_input.has_living_room else "")}
        {("- Kitchen" if budget_input.has_kitchen else "")}
        {("- Bedroom" if budget_input.has_bedroom else "")}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with product recommendations **available in India**.
        Use **Indian brands and pricing**. Include **search terms** suitable for Indian shopping platforms.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "lighting",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
            ]
        }}
```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """
    """
    # Ensure total costs stay within budget. Include search terms for each item to find on shopping websites like Flipkart, Amazon India, IKEA
    # Add shopping links for each item
    for category in result.get("budget_breakdown", []):
        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {
                    "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                    "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "ikea": f"https://www.ikea.com/in/en/search/?q={urllib.parse.quote_plus(search_terms)}",
                    "myntra": f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "ajio": f"https://www.ajio.com/search/?text={urllib.parse.quote_plus(search_terms)}"
                }
    return result
```

```
except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

- **/generate-party**: Suggests catering, venue, and decoration options for events

```
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party planning recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need party planning recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Party details:
        - Type: {budget_input.party_type}
        - Number of guests: {budget_input.num_guests}
        - Venue type: {budget_input.venue_type or "Not specified"}
        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}
        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}
        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in India using INR prices.
        Use Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "venue",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
            ]
        }}
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
```

```
    """
    "venue_suggestions": [
        {
            "name": "",
            "type": "",
            "capacity": 0,
            "estimated_cost": 0.0,
            "search_terms": ""
        }
    ],
    "remaining_budget": 0.0,
    "additional_suggestions": []
    """
```

Ensure all costs are in INR and total does not exceed the given budget.
 Provide search terms suitable for Indian websites such as BookMyShow, Swiggy, Flipkart, etc.
 """

```
    response = model.generate_content(prompt)
    result = extract_json_from_response(response.text)
```

```
    # Create INR calculation table
    result["calculation_table_inr"] = []
    categories = {}
```

```
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "Misc")
        cat_allocation = category.get("allocation", 0)
```

```
        if cat_name not in categories:
            categories[cat_name] = {
                "category": cat_name,
                "items_count": 0,
                "total_cost": 0,
                "percentage_of_budget": 0
            }
```

- linking the websites like amazon,flipkart,ikea ect.... to get the products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    "styling_tips": []
    )

    Make sure prices are in INR and stay within budget.
    Include Indian-friendly search terms for shopping.
    """
    response = model.generate_content([prompt, img])
else:
    # Only text input
    prompt = base_prompt + """

    Format the output as JSON:
    {
        "total_budget": 0.0,
        "jewelry_recommendations": [
            {
                "item_type": "",
                "description": "",
                "style": "",
                "estimated_price": 0.0,
                "search_terms": ""
            }
        ],
        "remaining_budget": 0.0,
        "styling_tips": []
    }

    Keep prices in INR and relevant to Indian brands.
    """
    response = model.generate_content(prompt)

    result = extract_json_from_response(response.text)

    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search terms", "")
```

- linking the websites like amazon,flipkart,ikea ect.... to get the products

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)

        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {}

                # Only add relevant platforms for this category
                if "amazon" in relevant_platforms:
                    item["shopping_links"]["amazon"] = f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}"
                if "flipkart" in relevant_platforms:
                    item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "bigbasket" in relevant_platforms:
                    item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/ps/?q={urllib.parse.quote_plus(search_terms)}"
                if "swiggy" in relevant_platforms:
                    item["shopping_links"]["swiggy"] = f"https://www.swiggy.com/search?query={urllib.parse.quote_plus(search_terms)}"
                if "zomato" in relevant_platforms:
                    item["shopping_links"]["zomato"] = f"https://www.zomato.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "bookmyshow" in relevant_platforms:
                    item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "myntra" in relevant_platforms:
                    item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "meesho" in relevant_platforms:
                    item["shopping_links"]["meesho"] = f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "google" in relevant_platforms:
                    item["shopping_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "booking" in relevant_platforms:
                    item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
                if "makemytrip" in relevant_platforms:
                    item["shopping_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(search_terms)}"
                if "oyorooms" in relevant_platforms:
                    item["shopping_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
                if "nobroker" in relevant_platforms:
                    item["shopping_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms)}"
```

```
    # Add search links for venue suggestions
    venue_platforms = ["google", "booking", "makemytrip", "oyorooms", "nobroker"]
    for venue in result.get("venue_suggestions", []):
        search_terms = venue.get("search_terms", "")
        if search_terms:
            venue["search_links"] = {}
            if "google" in venue_platforms:
                venue["search_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
            if "booking" in venue_platforms:
                venue["search_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
            if "makemytrip" in venue_platforms:
                venue["search_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(search_terms)}"
            if "oyorooms" in venue_platforms:
                venue["search_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
            if "nobroker" in venue_platforms:
                venue["search_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms)}"

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

- **/generate-jewelry**: Accepts text + image inputs to return style-matched jewelry options.

```
● async def jewelry_planner(request: Request, current_user:
  UserInDB = Depends(get_current_active_user)):
  """Jewelry budget planner page"""
```

```
● return templates.TemplateResponse("jewelry_planner.html",
    {"request": request, "user": current_user})
```

● get home_recommendations by using the function

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ]
            },
            "remaining_budget": 0.0,
            """

    except Exception as e:
        return {"error": str(e)}
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    for item in category.get("items", []):
        categories[cat_name]["items_count"] += 1
        categories[cat_name]["total_cost"] += item.get("estimated_price", 0)

    if result["total_budget"] > 0:
        categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

    for cat_data in categories.values():
        result["calculation_table_inr"].append(cat_data)

    # Define relevant shopping platforms for each category
    category_platforms = {
        "venue": ["google", "booking", "makemytrip", "oyorooms", "nobroker"],
        "catering": ["swiggy", "zomato"],
        "food": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
        "drinks": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
        "decoration": ["amazon", "flipkart", "meesho", "myntra"],
        "entertainment": ["bookmyshow", "amazon", "flipkart"],
        "gifts": ["amazon", "flipkart", "myntra", "meesho"],
        "photography": ["google", "amazon", "flipkart"],
        "music": ["amazon", "flipkart", "bookmyshow"],
        "games": ["amazon", "flipkart"],
        "accessories": ["amazon", "flipkart", "myntra", "meesho"],
        "transportation": ["makemytrip", "google"],
        "return_gifts": ["amazon", "flipkart", "myntra", "meesho"]
    }

    # Default platforms if category is not in our predefined list
    default_platforms = ["amazon", "flipkart", "google"]

    # Add shopping links for each item
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)

        for item in category.get("items", []):
```

- using the if condition to categorize the situation based products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search_terms", "")
        if search_terms:
            item["shopping_links"] = {
                "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "bluestone": f"https://www.bluestone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
            }
    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

Activity 2.3: Login and Register page routings

- Login page

/login: Authenticates user credentials and initiates a session upon successful login.

```
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})
```

- Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

```
@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

- Logout Page

/logout: Terminates the current user session and redirects to the login page.

```
@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
        except JWTError:
            pass

    # Use RedirectResponse instead of JSONResponse for proper redirection
    response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
    response.delete_cookie(key="access_token")
```

Activity 2.4: Ensure modular code structure

- **/token:** Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```
@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type": "bearer"})

    response.set_cookie(
        key="access_token",
        value=access_token, # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )

    return response
```

- **/session-info**: Retrieves metadata about the current user session, such as user ID and login status.
- **/session-data**: Returns detailed session-specific data used for personalization and recommendation tracking.

```
@app.get("/session-info")
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Get current user's session information"""
    if current_user.username in active_sessions:
        session = active_sessions[current_user.username]
        return {
            "username": session.username,
            "login_time": session.login_time,
            "last_activity": session.last_activity,
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60, # in minutes
            "user_data": session.user_data
        }
    else:
        raise HTTPException(status_code=404, detail="No active session found")

@app.post("/session-data")
async def update_session_data(
    data: Dict[str, Any],
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data": active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
# Fix for the /home-budget endpoint to properly handle the form submission
```

MILESTONE 3: Backend – FastAPI Integration (**main.py**) :

- **AI Bridge**: Connects the frontend interface to a Gemini-based AI engine.
- **Modular Design**: Provides API endpoints for Home, Party, and jewelry planners.
- **Key Features**: Includes CORS configuration, session handling, and structured input/output models for data reliability.

❖ Activity 3.1: Define FastAPI Routes

- **Home Planner (/generate-home)**: Accepts home-related preferences and budget to return tailored product suggestions.

```
@app.post("/home-budget")
✓ async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    ✓ if current_user.username in active_sessions:
    ✓     active_sessions[current_user.username].user_data["last_home_budget"] = {
        "timestamp": datetime.utcnow().isoformat(),
        "budget": budget_input.total_budget,
    ✓     "requirements": {
        "lights": budget_input.num_lights,
        "fans": budget_input.num_fans,
        "furniture": budget_input.num_furniture,
        "dining_tables": budget_input.num_dining_tables
        }
    }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    ✓ save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

```
# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```
# Update the jewelry-budget endpoint
@app.post("/jewelry-budget")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
        }

    # Get recommendations
    result = get_jewelry_recommendations(budget_input, image_path)

    # Save to history with image info
    input_data = budget_input.dict()
    if image:
        input_data["image"] = image.filename

    save_to_history(
        username=current_user.username,
        recommendation_type="jewelry",
        input_data=input_data,
        result=result
    )

    return result
@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:
```

```
    return {"history": []}

# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

- User Auth ([/register](#), [/login](#), [/logout](#), [/token](#)): Manages user registration, authentication, and session control.
- Session Info ([/session-info](#), [/session-data](#)): Retrieves user-specific session data for personalized AI interaction.